
3.7 Exercises

Reinforcement

- R-3.1 Give the next five pseudorandom numbers generated by the process described on page 113, with $a = 12$, $b = 5$, and $n = 100$, and 92 as the seed for `cur`.
- R-3.2 Write a Java method that repeatedly selects and removes a random entry from an array until the array holds no more entries.
- R-3.3 Explain the changes that would have to be made to the program of Code Fragment 3.8 so that it could perform the Caesar cipher for messages that are written in an alphabet-based language other than English, such as Greek, Russian, or Hebrew.
- R-3.4 The `TicTacToe` class of Code Fragments 3.9 and 3.10 has a flaw, in that it allows a player to place a mark even after the game has already been won by someone. Modify the class so that the `putMark` method throws an `IllegalStateException` in that case.
- R-3.5 The `removeFirst` method of the `SinglyLinkedList` class includes a special case to reset the tail field to **null** when deleting the last node of a list (see lines 51 and 52 of Code Fragment 3.15). What are the consequences if we were to remove those two lines from the code? Explain why the class would or would not work with such a modification.
- R-3.6 Give an algorithm for finding the second-to-last node in a singly linked list in which the last node is indicated by a null next reference.
- R-3.7 Consider the implementation of `CircularlyLinkedList.addFirst`, in Code Fragment 3.16. The else body at lines 39 and 40 of that method relies on a locally declared variable, `newest`. Redesign that clause to avoid use of any local variable.
- R-3.8 Describe a method for finding the middle node of a doubly linked list with header and trailer sentinels by “link hopping,” and without relying on explicit knowledge of the size of the list. In the case of an even number of nodes, report the node slightly left of center as the “middle.” What is the running time of this method?
- R-3.9 Give an implementation of the `size()` method for the `SingularlyLinkedList` class, assuming that we did not maintain `size` as an instance variable.
- R-3.10 Give an implementation of the `size()` method for the `CircularlyLinkedList` class, assuming that we did not maintain `size` as an instance variable.
- R-3.11 Give an implementation of the `size()` method for the `DoublyLinkedList` class, assuming that we did not maintain `size` as an instance variable.
- R-3.12 Implement a `rotate()` method in the `SinglyLinkedList` class, which has semantics equal to `addLast(removeFirst())`, yet without creating any new node.

- R-3.13 What is the difference between a shallow equality test and a deep equality test between two Java arrays, A and B , if they are one-dimensional arrays of type `int`? What if the arrays are two-dimensional arrays of type `int`?
 - R-3.14 Give three different examples of a single Java statement that assigns variable, backup, to a new array with copies of all `int` entries of an existing array, original.
 - R-3.15 Implement the `equals()` method for the `CircularlyLinkedList` class, assuming that two lists are equal if they have the same sequence of elements, with corresponding elements currently at the front of the list.
 - R-3.16 Implement the `equals()` method for the `DoublyLinkedList` class.
-

Creativity

- C-3.17 Let A be an array of size $n \geq 2$ containing integers from 1 to $n - 1$ inclusive, one of which is repeated. Describe an algorithm for finding the integer in A that is repeated.
- C-3.18 Let B be an array of size $n \geq 6$ containing integers from 1 to $n - 5$ inclusive, five of which are repeated. Describe an algorithm for finding the five integers in B that are repeated.
- C-3.19 Give Java code for performing `add( $e$ )` and `remove( $i$ )` methods for the `Scoreboard` class, as in Code Fragments 3.3 and 3.4, except this time, don't maintain the game entries in order. Assume that we still need to keep n entries stored in indices 0 to $n - 1$. You should be able to implement the methods without using any loops, so that the number of steps they perform does not depend on n .
- C-3.20 Give examples of values for a and b in the pseudorandom generator given on page 113 of this chapter such that the result is not very random looking, for $n = 1000$.
- C-3.21 Suppose you are given an array, A , containing 100 integers that were generated using the method `r.nextInt(10)`, where r is an object of type `java.util.Random`. Let x denote the product of the integers in A . There is a single number that x will equal with probability at least 0.99. What is that number and what is a formula describing the probability that x is equal to that number?
- C-3.22 Write a method, `shuffle( $A$ )`, that rearranges the elements of array A so that every possible ordering is equally likely. You may rely on the `nextInt( $n$ )` method of the `java.util.Random` class, which returns a random number between 0 and $n - 1$ inclusive.
- C-3.23 Suppose you are designing a multiplayer game that has $n \geq 1000$ players, numbered 1 to n , interacting in an enchanted forest. The winner of this game is the first player who can meet all the other players at least once (ties are allowed). Assuming that there is a method `meet( $i, j$ )`, which is called each time a player i meets a player j (with $i \neq j$), describe a way to keep track of the pairs of meeting players and who is the winner.

- C-3.24 Write a Java method that takes two three-dimensional integer arrays and adds them componentwise.
- C-3.25 Describe an algorithm for concatenating two singly linked lists L and M , into a single list L' that contains all the nodes of L followed by all the nodes of M .
- C-3.26 Give an algorithm for concatenating two doubly linked lists L and M , with header and trailer sentinel nodes, into a single list L' .
- C-3.27 Describe in detail how to swap two nodes x and y (and not just their contents) in a singly linked list L given references only to x and y . Repeat this exercise for the case when L is a doubly linked list. Which algorithm takes more time?
- C-3.28 Describe in detail an algorithm for reversing a singly linked list L using only a constant amount of additional space.
- C-3.29 Suppose you are given two circularly linked lists, L and M . Describe an algorithm for telling if L and M store the same sequence of elements (but perhaps with different starting points).
- C-3.30 Given a circularly linked list L containing an even number of nodes, describe how to split L into two circularly linked lists of half the size.
- C-3.31 Our implementation of a doubly linked list relies on two sentinel nodes, header and trailer, but a single sentinel node that guards both ends of the list should suffice. Reimplement the `DoublyLinkedList` class using only one sentinel node.
- C-3.32 Implement a circular version of a doubly linked list, without any sentinels, that supports all the public behaviors of the original as well as two new update methods, `rotate()` and `rotateBackward()`.
- C-3.33 Solve the previous problem using inheritance, such that a `DoublyLinkedList` class inherits from the existing `CircularlyLinkedList`, and the `DoublyLinkedList.Node` nested class inherits from `CircularlyLinkedList.Node`.
- C-3.34 Implement the `clone()` method for the `CircularlyLinkedList` class.
- C-3.35 Implement the `clone()` method for the `DoublyLinkedList` class.

Projects

- P-3.36 Write a Java program for a matrix class that can add and multiply arbitrary two-dimensional arrays of integers.
- P-3.37 Write a class that maintains the top ten scores for a game application, implementing the add and remove methods of Section 3.1.1, but using a singly linked list instead of an array.
- P-3.38 Perform the previous project, but use a doubly linked list. Moreover, your implementation of `remove(i)` should make the fewest number of pointer hops to get to the game entry at index i .
- P-3.39 Write a program that can perform the Caesar cipher for English messages that include both upper- and lowercase characters.

- P-3.40 Implement a class, SubstitutionCipher, with a constructor that takes a string with the 26 uppercase letters in an arbitrary order and uses that as the encoder for a cipher (that is, A is mapped to the first character of the parameter, B is mapped to the second, and so on.) You should derive the decoding map from the forward version.
- P-3.41 Redesign the CaesarCipher class as a subclass of the SubstitutionCipher from the previous problem.
- P-3.42 Design a RandomCipher class as a subclass of the SubstitutionCipher from Exercise P-3.40, so that each instance of the class relies on a random permutation of letters for its mapping.
- P-3.43 In the children's game, Duck, Duck, Goose, a group of children sit in a circle. One of them is elected "it" and that person walks around the outside of the circle. The person who is "it" pats each child on the head, saying "Duck" each time, until randomly reaching a child that the "it" person identifies as "Goose." At this point there is a mad scramble, as the "Goose" and the "it" person race around the circle. Whoever returns to the Goose's former place first gets to remain in the circle. The loser of this race is the "it" person for the next round of play. The game continues like this until the children get bored or an adult tells them it's snack time. Write software that simulates a game of Duck, Duck, Goose.

Chapter Notes

The fundamental data structures of arrays and linked lists discussed in this chapter belong to the folklore of computer science. They were first chronicled in the computer science literature by Knuth in his seminal book on *Fundamental Algorithms* [60].